Laboratorio No.1 – Semana 14 - Integración servicios en la nube

Nombre: Alejandro Mogollón Rodriguez **Carné**: 1132125

Objetivo: Desarrollar y publicar una aplicación web que permita interactuar con archivos almacenados en Azure Blob Storage, utilizando control de versiones con Git y una pipeline (script) CI/CD de Azure DevOps para automatizar el despliegue de la aplicación en Azure App Service

Resultado esperado

- Aplicación web funcional ejecutándose en Azure App Service.
- Conexión entre App Service y Azure Blob Storage.
- Listado y consulta de archivos almacenados en un contenedor Blob.
- Operación de carga de archivos al Blob Storage.
- Repositorio Git con historial de cambios.
- Pipeline de Azure DevOps que compile y despliegue automáticamente la aplicación.
- Evidencias que demuestren el funcionamiento de la solución.

Repositorio recomendado
Utilice como punto de partida el ejemplo oficial de Microsoft/Azure Samples revisado para esta actividad:
GitHub: https://github.com/Azure-Samples/serviceconnector-webapp-storageblob-dotnet
El tutorial oficial utiliza una aplicación web .NET conectada a Azure Blob Storage mediante una identidad administrada y Service Connector. La documentación indica además que la aplicación puede desplegarse en Azure App Service.

Tecnologías

Tecnología	Uso
C# / .NET	Desarrollo de la aplicación web.
Azure Blob Storage	Almacenamiento de archivos.
Azure App Service	Hospedaje de la aplicación.
Git	Control de versiones.
Azure DevOps Pipelines	Integración y despliegue continuo.
Managed Identity / Service Connector	Acceso seguro desde App Service al almacenamiento.

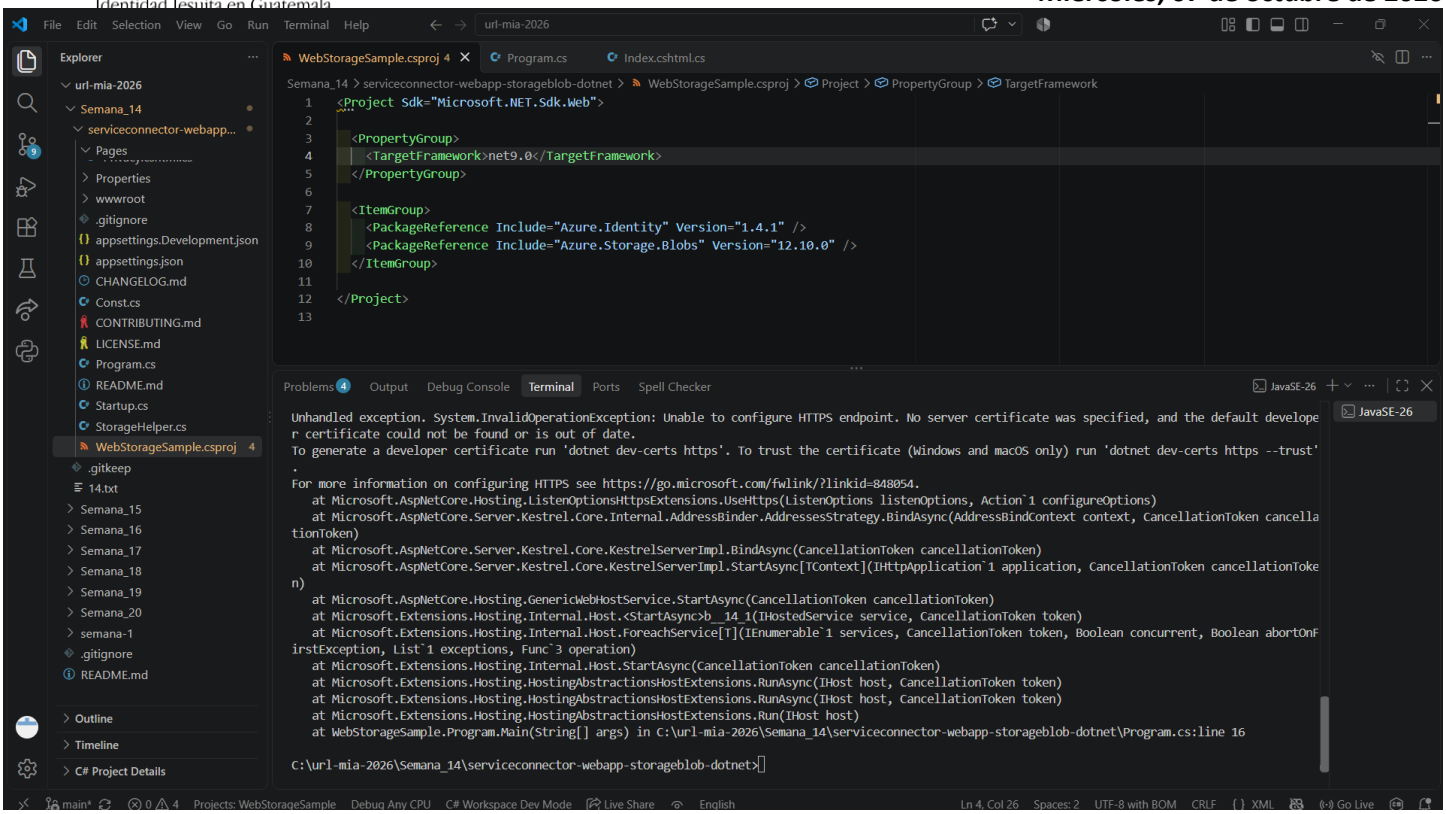
Realización de la actividad

1. Preparar el repositorio
En el repositorio brindado arriba, realice un fork del repositorio base o copie el proyecto a un repositorio propio. El repositorio debe pertenecer al equipo de trabajo. Realice un primer commit identificable

---

2. Revisar la aplicación:
Ejecute la aplicación localmente y revise cómo obtiene acceso al Blob Storage. Identifique el punto del código donde se realiza la lectura/escritura de blobs y documente brevemente su funcionamiento. Visual Studio -> carpeta -> MIA_Semana14 -> git clone [repo url] -> dotnet commands

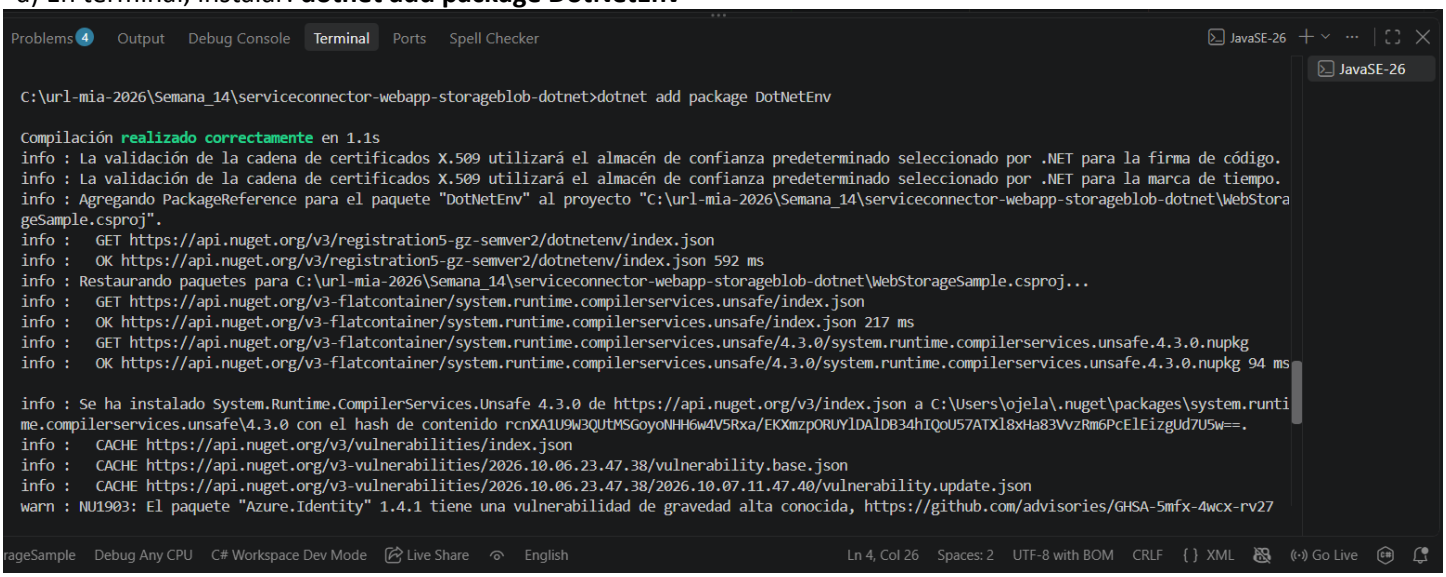
NOTA: cambia ren .csproj file -> <TargetFramework>net9.0</TargetFramework>



---

3. Crear DotNet Env

a) En terminal, instalar: **dotnet add package DotNetEnv**

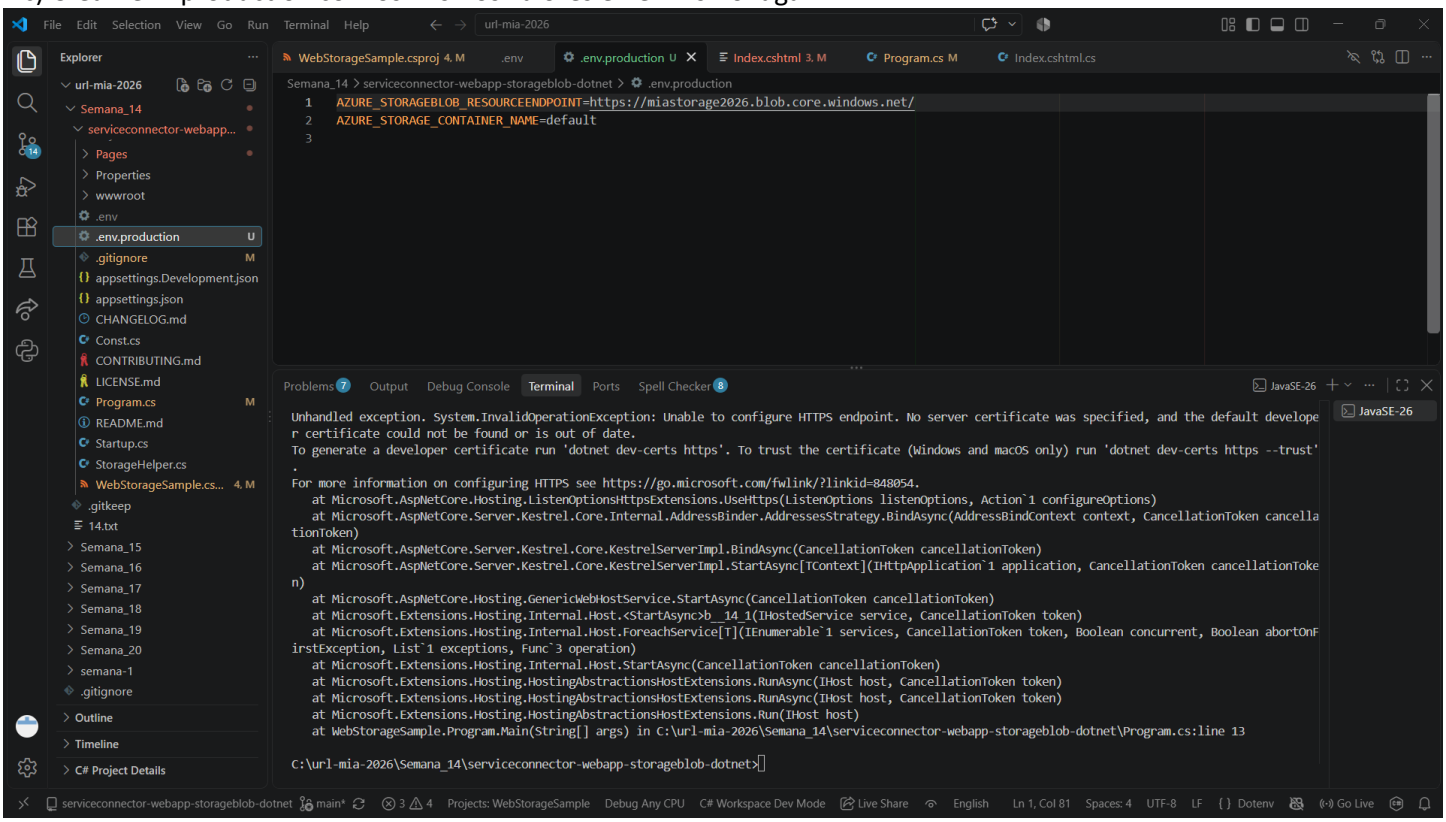


b) Crear .env file en raíz de proyecto, valores ->

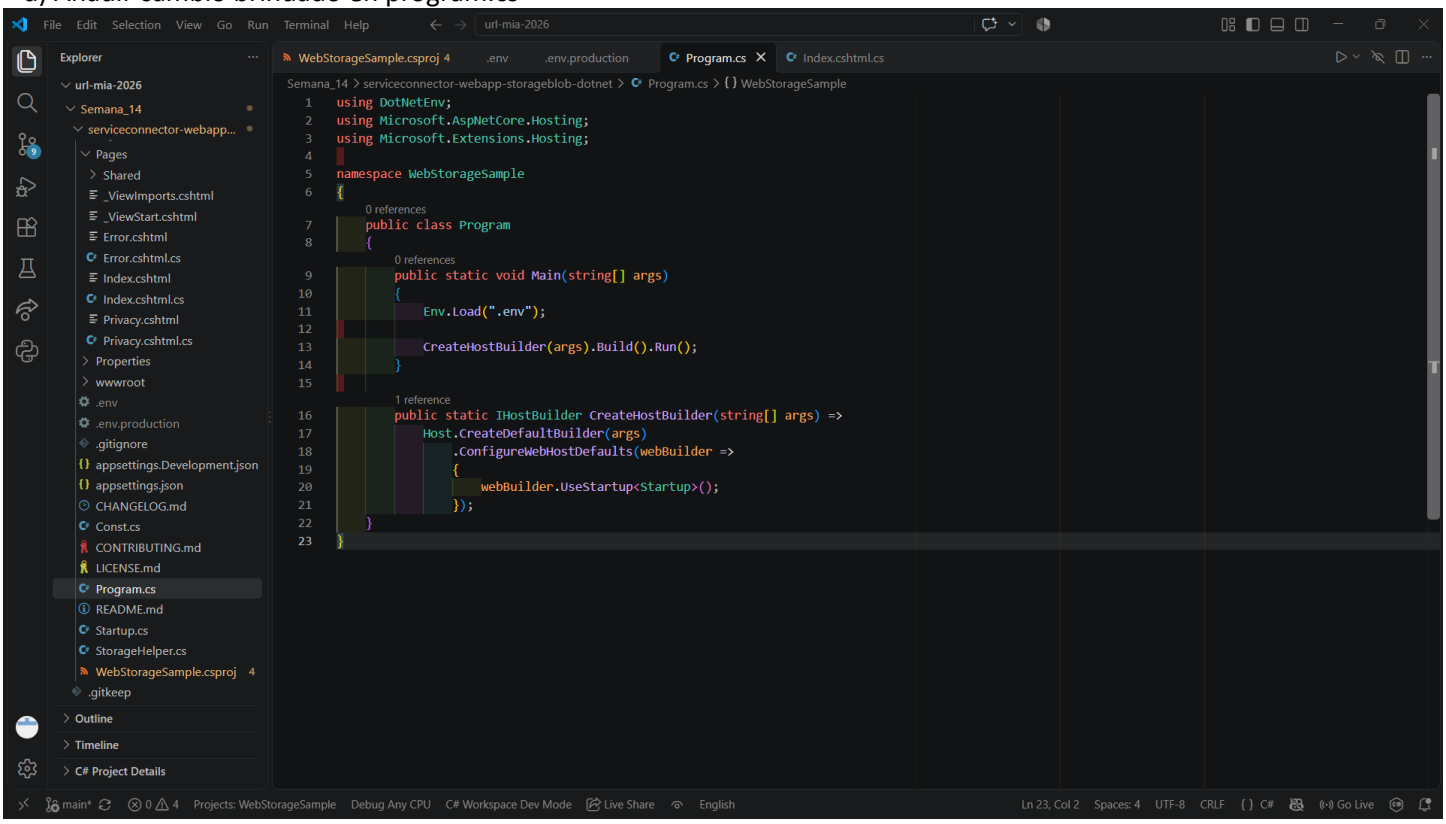
AZURE_STORAGEBLOB_RESOURCEENDPOINT=https://miarchivos2026.blob.core.windows.net/

AZURE_STORAGE_CONTAINER_NAME=default

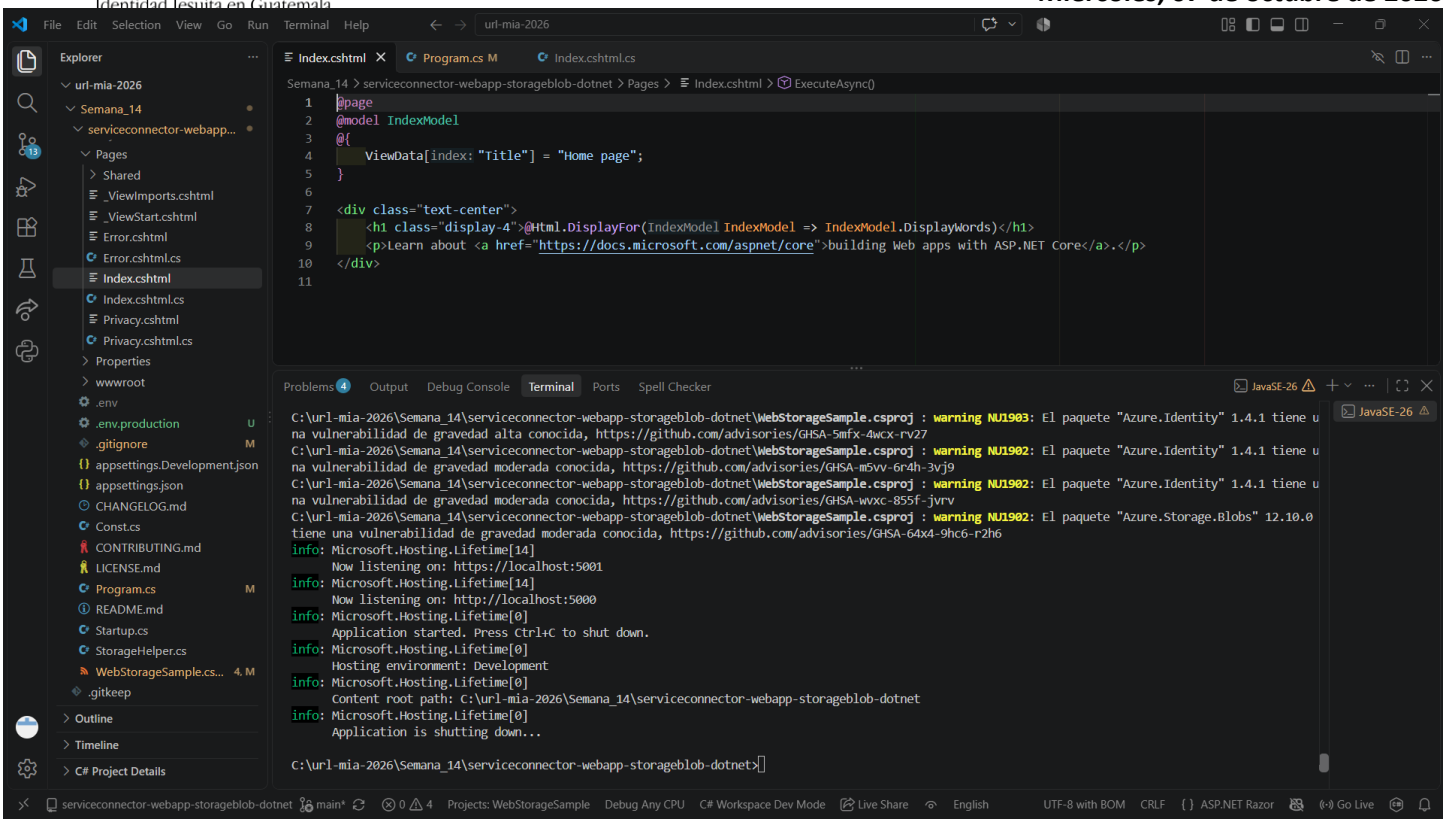
c) Crear .env.production con los mismos valores en el mismo lugar



d) Anadir cambio brindado en program.cs

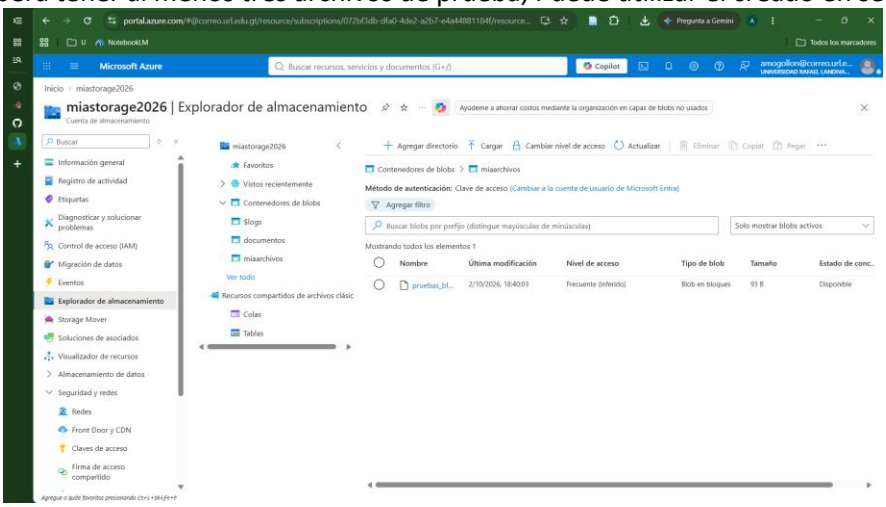


e) dotnet clean -> dotnet restore -> dotnet build -> dotnet run



4. Crear recursos en Azure

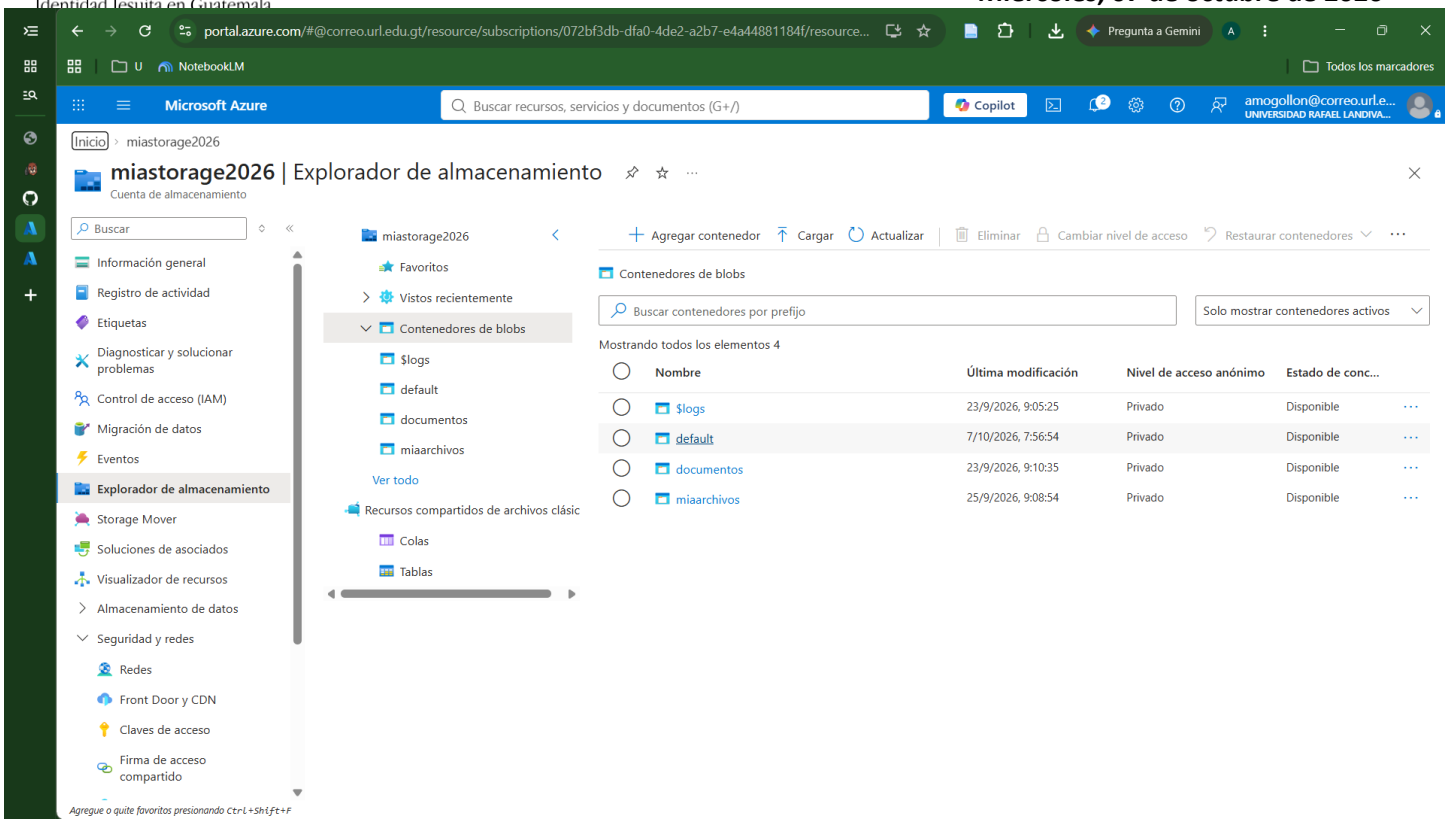
En azure portal site, cree o utilice un Resource Group, una Storage Account con Blob Storage y un contenedor. El contenedor deberá tener al menos tres archivos de prueba, Puede utilizar el creado en semana 13.



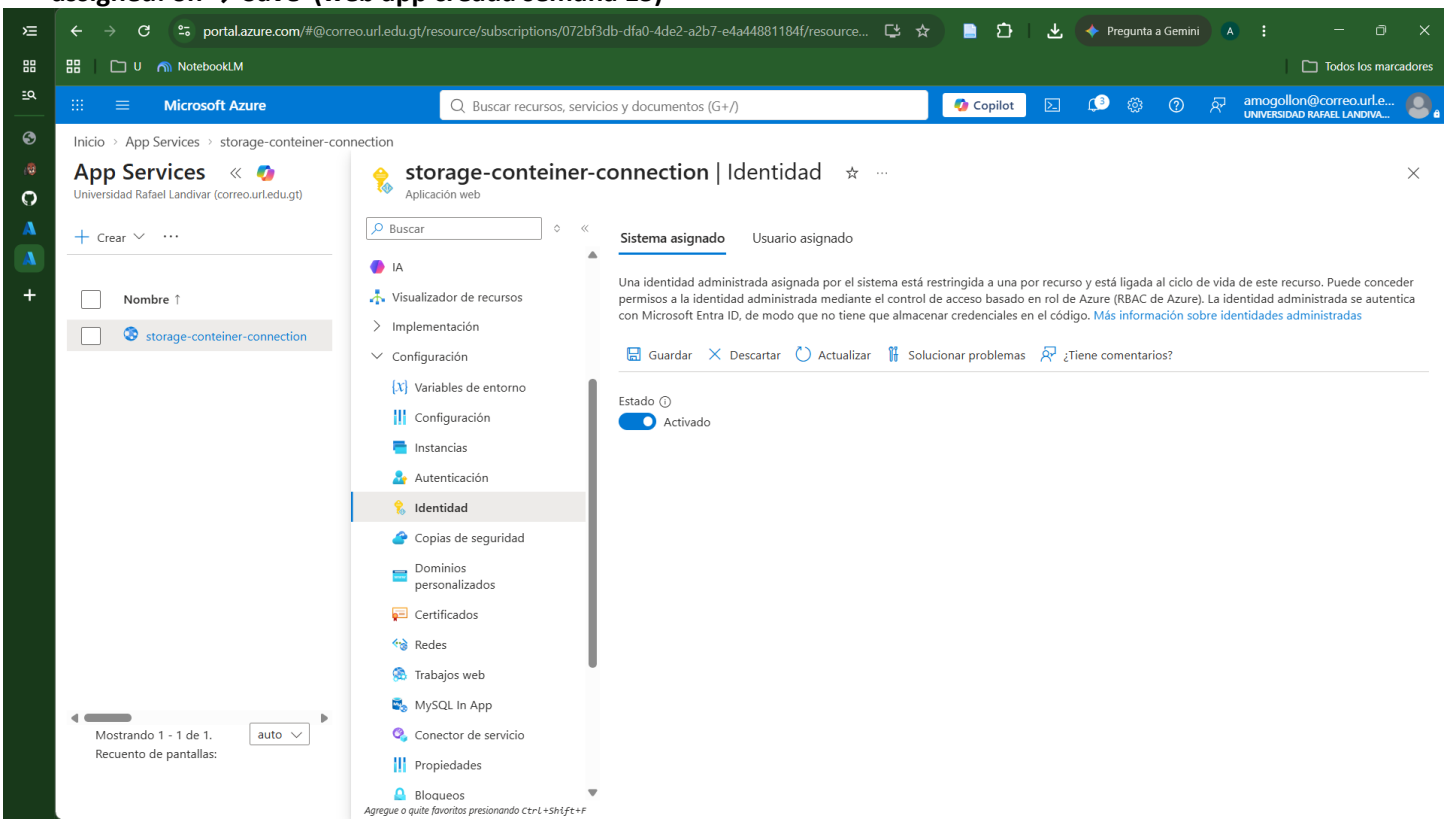
5. Conectar App Service con Blob Storage

Configure la conexión entre App Service y Blob Storage mediante Service Connector utilizando identidad administrada. Verifique que la identidad tenga los permisos necesarios sobre los blobs.

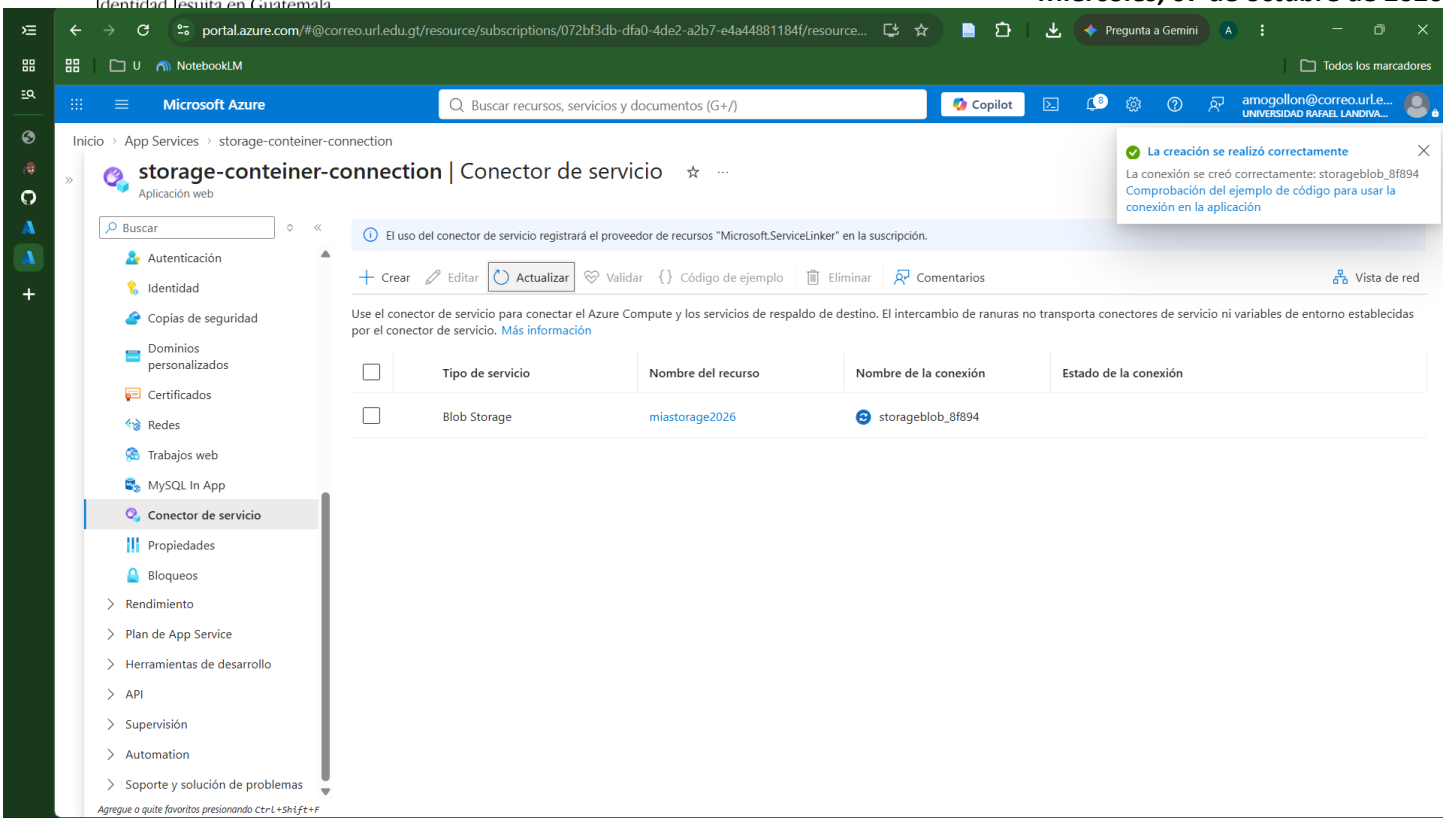
a) Azure Portal → **Storage accounts → tu Storage Account → Data storage → Containers**
b) Crear contenedor “default



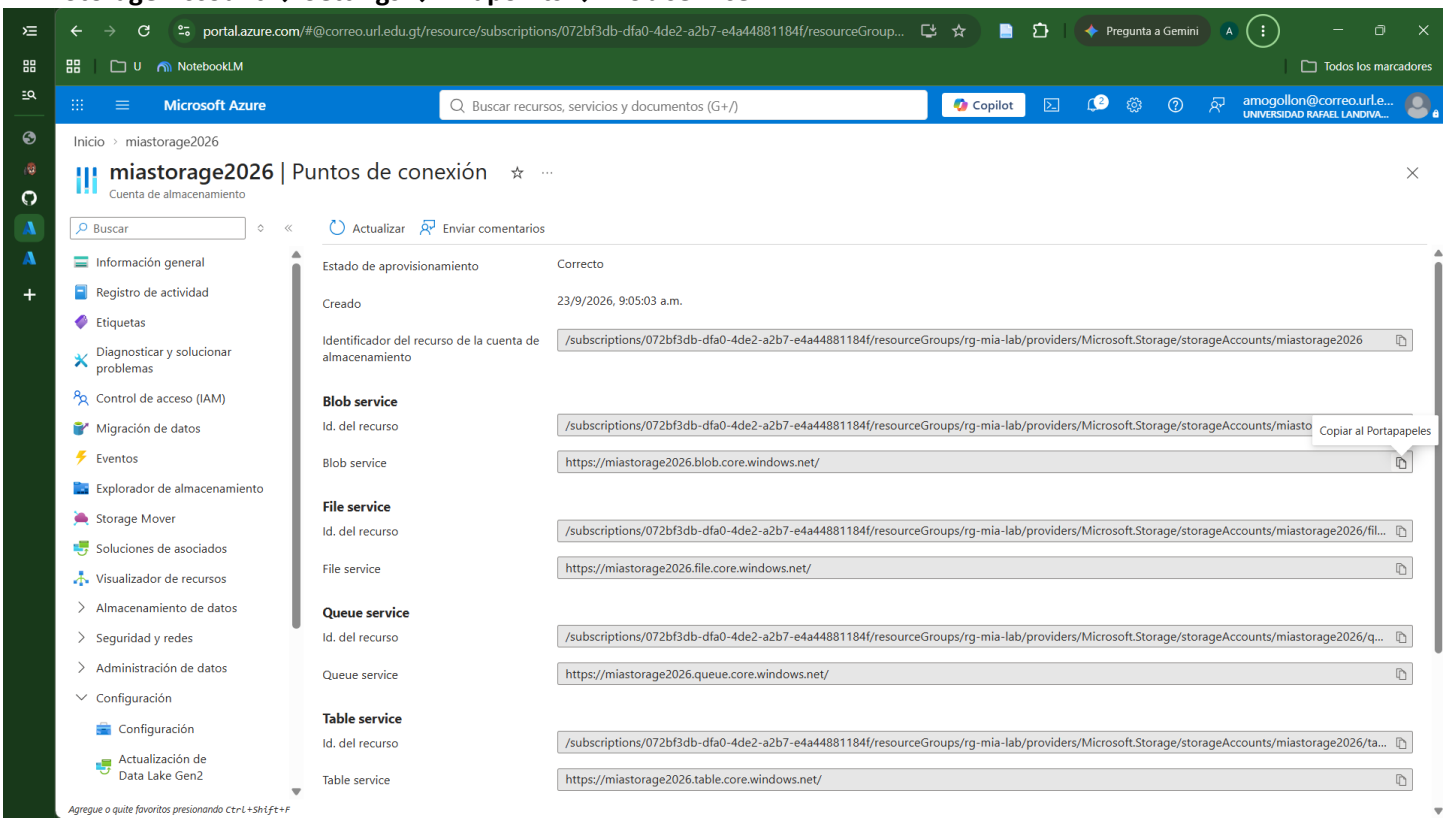
c) Activa identidad administrada en App Service: **Azure Portal → App Service → Settings → Identity → System assigned: on → Save (web app creada semana 13)**



d) Configurar la conexión entre App Service y Blob Storage: **App Service → Settings → Service Connector → Create → Seleccionar "Storage Blob" → Seleccionar Storage Account [tu_blob_name] → Authentication "System-assigned managed identity" → Review + Create**.



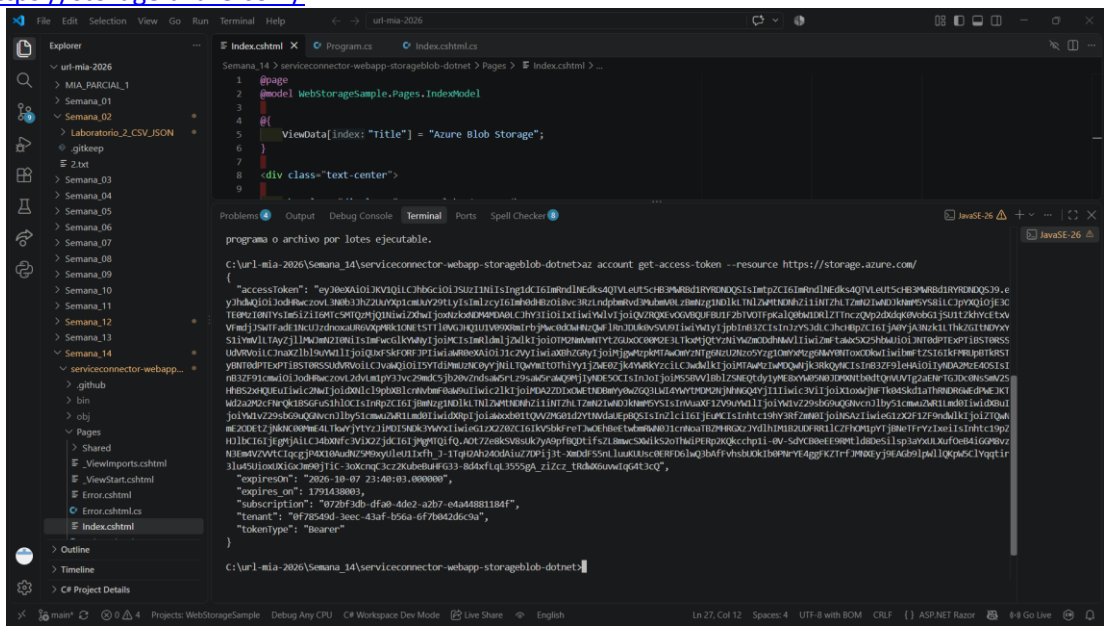
e) Reemplazar storage blob endpoint y container nombre en .env file
Storage Account → Settings → Endpoints → Blob service



6. Probar el acceso al almacenamiento:

Ejecute la aplicación local y compruebe que puede leer los blobs. Realice al menos una operación de carga de archivo y verifique desde Azure Portal que el archivo llegó al contenedor. Visual Studio -> dotnet run

a) Verifique que Azure CLI esté instalado ejecutando en la terminal de VS Code: az --version
b) Inicie sesión en Azure: az login
c) Verifique la cuenta de Azure activa: az account show
d) Verifique que pueda obtener un token para Azure Storage: az account get-access-token --resource https://storage.azure.com/

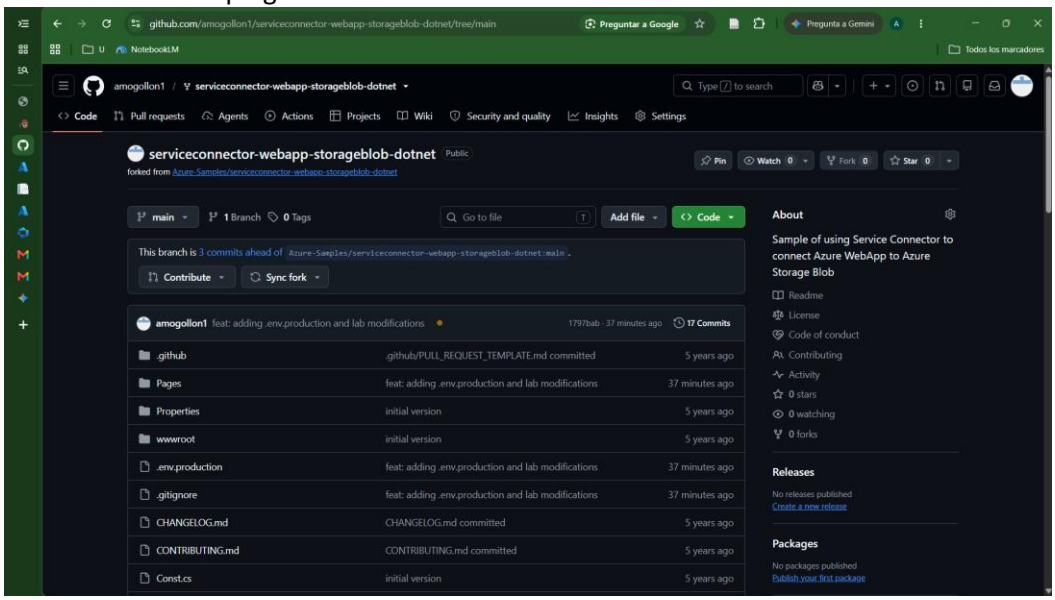


e) Si se muestra un accessToken la autenticación esta correcta, ejecutar -> dotnet run
f) Si hay error en acceso, ejecutar en terminarl vs code: az role assignment create --assignee "TU_CORREO_DE_AZURE" --role "Storage Blob Data Contributor" --scope "/subscriptions/TU_SUBSCRIPTION_ID/resourceGroups/TU_RESOURCE_GROUP/providers/Microsoft.Storage/storageAccounts/TU_STORAGE"

Obtener info: az account show --query "{subscriptionId:id, user:user.name}" -o table
az storage account show --name “tu_storage” --query "{id:id, resourceGroup:resourceGroup}" -o table

g) Cerrar process ctr +c y logear de nuevo: az logout y az login, Por ultimo dotnet run

Nota: no olvidar subir cambios a repo github fork



7. Crear Pipeline Yml

En la raíz de tu fork de GitHub crear archivo “azure-pipelines.yml”

Colocar en el archivo lo siguiente:

```
trigger:
- main

pool:
  vmImage: 'windows-latest'

variables:
  buildConfiguration: 'Release'
  dotnetVersion: '9.x'

  # Completar posteriormente
  azureSubscription: 'NOMBRE-DE-TU-SERVICE-CONNECTION'
  appName: 'NOMBRE-DE-TU-APP-SERVICE'

steps:

# 1. Instalar .NET 9
- task: UseDotNet@2
  displayName: 'Instalar .NET 9 SDK'
  inputs:
    packageType: 'sdk'
    version: '$(dotnetVersion)'

# 2. Restaurar dependencias
- task: DotNetCoreCLI@2
  displayName: 'Restaurar dependencias'
  inputs:
    command: 'restore'
    projects: '**/*.csproj'

# 3. Configurar ambiente de producción
- powershell: |
    Copy-Item ".env.production" ".env" -Force
  displayName: 'Configurar .env de producción'

# 4. Compilar
- task: DotNetCoreCLI@2
  displayName: 'Compilar aplicación'
  inputs:
    command: 'build'
    projects: '**/*.csproj'
    arguments: '--configuration $(buildConfiguration) --no-restore'

# 5. Publicar
- task: DotNetCoreCLI@2
  displayName: 'Publicar aplicación'
  inputs:
    command: 'publish'
```

```
    projects: '**/*.csproj'
    arguments: '--configuration $(buildConfiguration) --output $(Build.ArtifactStagingDirectory)/publish --no-build'
    zipAfterPublish: true

  # 6. Desplegar en Azure App Service
  - task: AzureWebApp@1
    displayName: 'Desplegar en Azure App Service'
    inputs:
      azureSubscription: '$(azureSubscription)'
      appType: 'webApp'
      appName: '$(appName)'
      package: '$(Build.ArtifactStagingDirectory)/publish/**/*.zip'
```

---

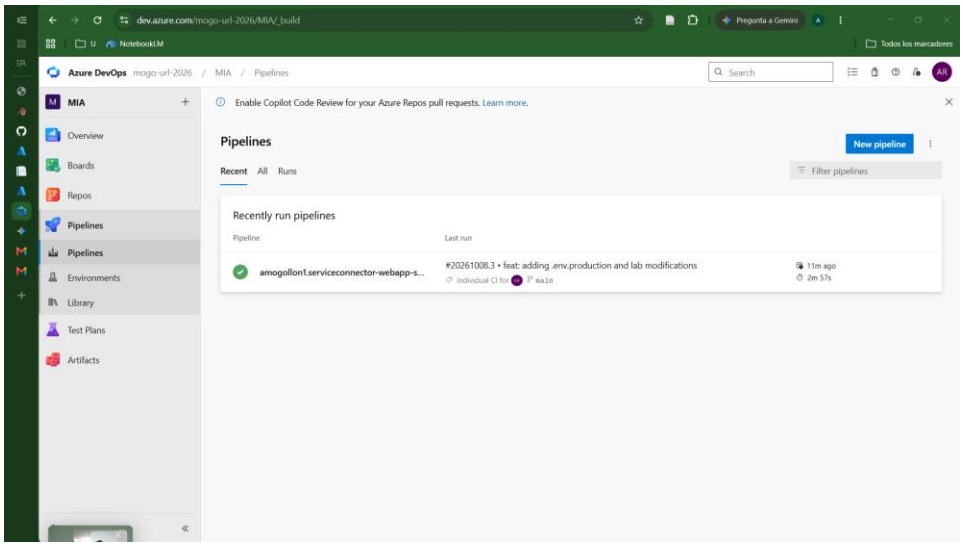
8. Crear pipeline CI/CD

Cree un pipeline YAML que restaure dependencias, compile, publique y despliegue la aplicación en Azure App Service.

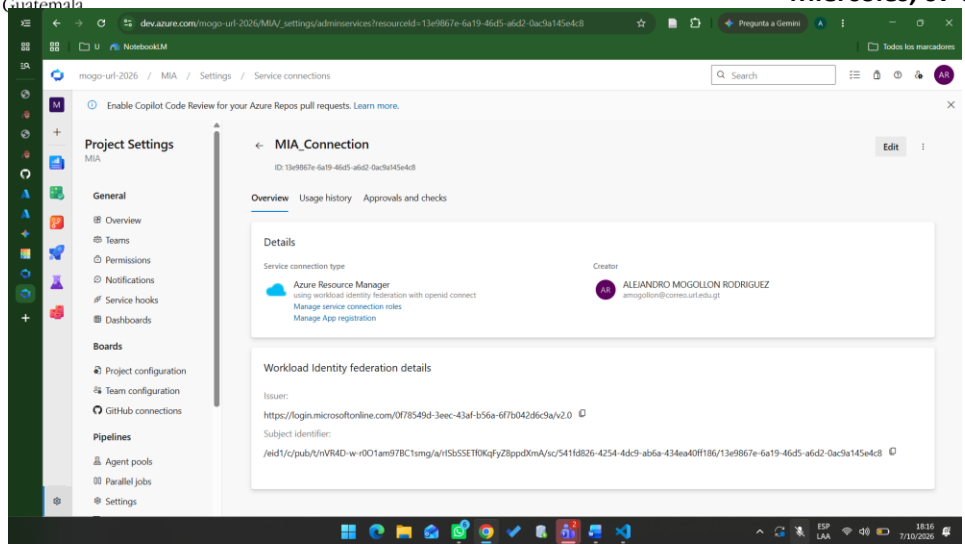
a) Conectarse a Azure Devops son misma cuenta que azure portal -> https://aex.dev.azure.com/me

b) Crear pipeline: **En Azure devOps -> Menu izquierda -> pipelines -> Create Pipeline -> Seleccionar Github -> Autoriza Azure DevOps para acceder a GitHub -> seleccionar repositorio que hizo fork -> seleccionar Existing Azure Pipelines YAML file -> Branch: main Path: /azure-pipelines.yml**

c) Probar seleccionando “run”



d) Conectar Azure DevOps con Azure: **Project Settings → Service connections → New service connection -> Azure Resource Manager -> luego, Workload identity federation -> Selecciona tu suscripción de Azure -> y resource group donde estan tus recursos -> Anadir nombre “MIA_Connection” -> Seleccionar “grant access permission to all pipelines” -> Save**



---

9. Conectar Pipeline con App

a) Obtener nombre app: En Azure Portal App Services → tu aplicación → Copia nombre app

b) en github en tu "azure-pipelines.yml" file busca: azureSubscription y appName, cambiar por valores reales.

azureSubscription: TU_Service_Connection_name
appName: Tu _app_name

c) Al hacer commit, verificar el pipeline se ejecuta automáticamente

---

10. Validar la solución

Compruebe desde la URL pública que la aplicación funciona y que el acceso al Blob Storage continúa operativo después del despliegue.

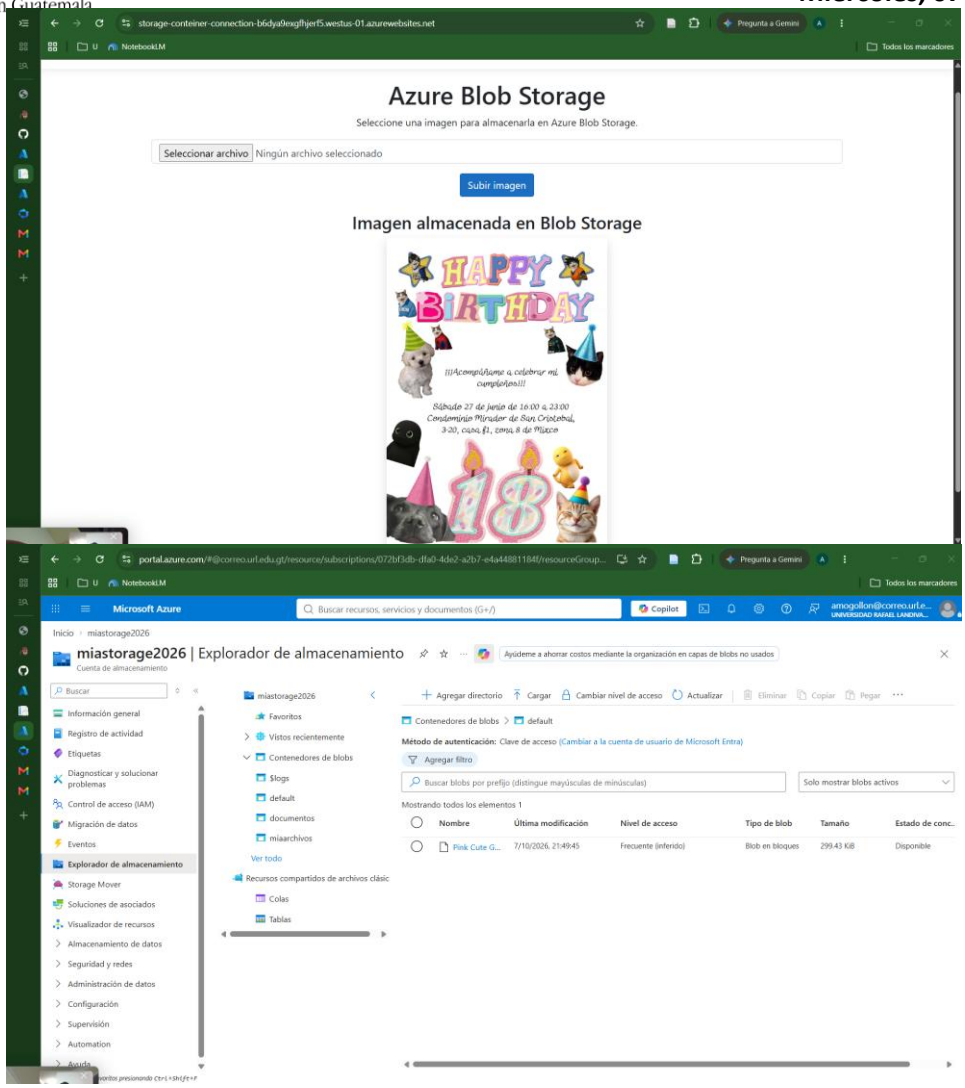
a) acceso de la aplicación al Blob: Azure Portal → App Service → Settings → Environment variables → App settings -> colocar AZURE_STORAGEBLOB_RESOURCEENDPOINT
https://tu_storage.blob.core.windows.net/
(si existe obviar)

b) Verificar app: Azure Portal → App Service → OverView → Default Domain → click en URL publica

c) En la app publicada realizar cambios en archivos

d) Verificar que los cambios se representen en storage account: Azure Portal → Storage Account → Storage Browser → Blob containers → Seleccionar su container → Verificar cambios

Resultado de imagen de prueba publicada desde web



---

11. Preguntas de analisis

1. ¿Para qué sirve el archivo .env en la aplicación?
   El archivo .env se encarga de almacenar las variables de ambiente fuera del código fuente para evitar incluir credenciales al repositorio de git y facilita cambiar la configuración del endpoint o contenedores en caso de ser necesario.
2. ¿Por qué utilizar Blob Storage y no guardar los archivos dentro del proyecto?
   Porque los archivos en Blob Storage, al estar en la nube, son independientes a lo que suceda en la máquina local, es decir se mantienen publicados (a menos que se eliminen a propósito), además de ser un almacenamiento con bastante escalabilidad.
3. ¿Cuál es la diferencia entre el endpoint de Blob Storage y la URL de App Service?
   El endpoint es la dirección de la API de Azure y se usa para leer, subir y gestionar los archivos. Mientras que la URL de App Service es el dominio público donde se muestra la interfaz html del .NET.
4. ¿Por qué utilizar una identidad administrada en lugar de una Storage Account Key?
   Es un tema de seguridad, ya que elimina la necesidad de colocar secret keys o archivos de configuración en el

código, gracias a que Azure se encarga de autenticar el App Service ante el Storage Account.

5. ¿Qué función cumplen GitHub, Azure DevOps y la Service Connection?
   GitHub aloja el código fuente y el historial de cambios.
   Azure DevOps ejecuta el pipeline que se encarga de compilar, probar y desplegar la aplicación.
   Service Connection es el puente de autenticación entre Azure DevOps y Azure Portal.
6. ¿Qué ocurre después de hacer push a main?
   Azure DevOps recibe una notificación debido a los cambios, haciendo que el pipeline se ejecute nuevamente de manera automática, descarga el nuevo código, lo restaura, compila y despliega en Azure App Service.
7. Si el pipeline termina correctamente pero la aplicación recibe un 403 al subir un archivo, ¿qué revisarías?
   Debido a que este error se refiere a algún problema de permisos o autorización, revisaría todas las conexiones que se realizaron en este laboratorio, iniciando por las que son directamente en Azure, asegurarme que sea el contenedor correcto, que el connector esté bien configurado y así sucesivamente.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-14/lab1/ y coloque en ella un PDF con la solución del laboratorio.
Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.
Enviar link de github a plataforma de la universidad.